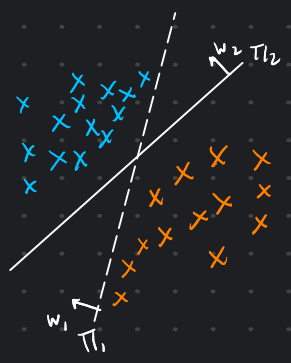
॥श्री गणेशाय नमः॥

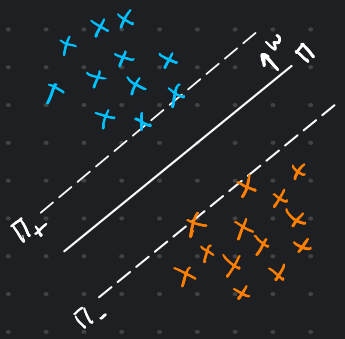
Support Vector Machines



Consider some linearly separable data.
We could draw multiple hyperplanes to separate the classes.

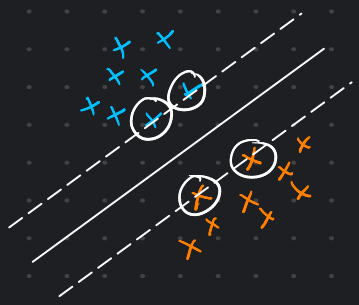
consider π_1.
It has many points close by.
Slight changes would affect the results.
Also P that the nearby points belong to either of the classes = $P(y=1) \approx 0.5$

Key idea:
We want a π that separates the +ve and the -ve points as widely as possible.
π: margin maximizing hyperplane.



Say we go parallel to π in the positive direction to get hyperplane π+ and in the -ve direction to get π- when we touch the 1st +ve and 1st -ve points respectively.

$\text{margin} = d(\pi_+, \pi_-)$ } SVM tries to maximize this.



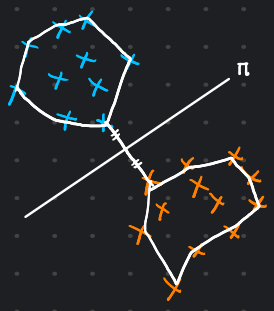
Support vectors:
These are the points that touch π+ and π-.

Alternative geometric interpretation:

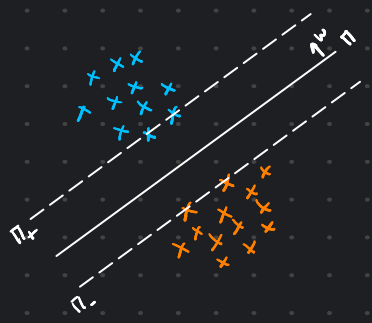
Convex hull: The smallest convex polygon for which all points lie either inside it or on it.



Idea:
Build convex hulls for the classes separately.
Then find the shortest line connecting these hulls.
The hyperplane that is the perpendicular bisector of that line is the maximum margin classifier.



Mathematical formulation



$\pi : w^T x + b = 0$

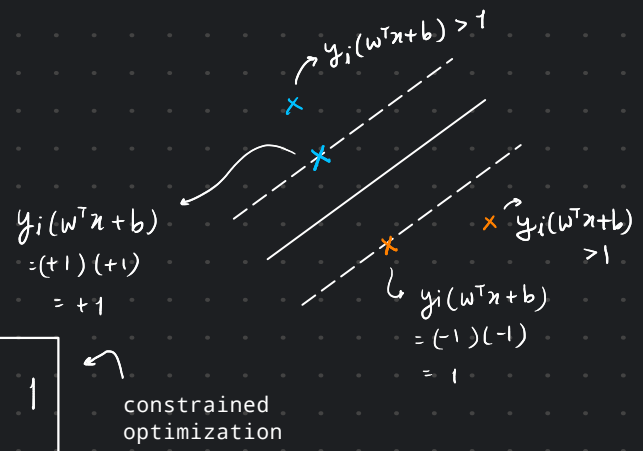
$\pi^+ : w^T x + b = -1$

$\pi^- : w^T x + b = -1$

$\text{margin} = d(\pi^+, \pi^-) = \frac{2}{\|w\|}$

w will be perpendicular to π, π_+, π_-.

$$(w^*, b^*) = \underset{w,b}{\text{argmax}} \frac{2}{\|w\|} \quad \text{s.t.} \quad y_i(w^T x + b) \geq 1 \text{ for all } x_i\text{'s}$$



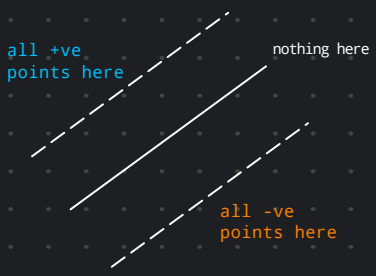
i.e.

$$\boxed{(w^*, b^*) = \underset{w,b}{\text{argmax}} \frac{2}{\|w\|} \quad \text{s.t.} \quad \forall i,\ y_i(w^T x_i + b) \geq 1}$$

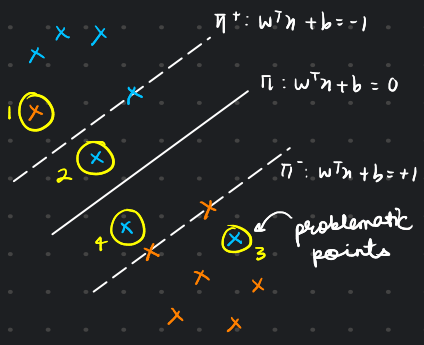
constrained optimization

But this works well only if the points are linearly separable.

This is called a hard-margin classifier. We are specifying exact conditions that aren't to be violated.



Consider the problematic points.



For point 4: $y_i(w^T x_i + b) \approx -0.5$

$= y_i(w^T x + b) = 1 - 1.5$

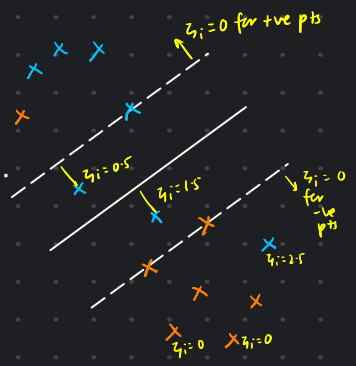
For point 3: $y_i(w^T x_i + b) \approx -1.5$

$y_i(w^T x_i + b) = 1 - 2.5$ → ζ_i

For point 2: $y_i(w^T x + b) = 0.5$

$= 1 - 0.5$ → ζ_i

Represented as $1 - \zeta$



We define ζ_i such that it is zero for unproblematic points.
It increases as a point goes further away from the correct π in the incorrect direction.

$\zeta_i = 0$ if a point is correctly classified
i.e. $y_i(w^T x_i + b) \geq 1$

$\zeta_i > 0$ and it is equivalent to some units of distance away from the correct hyperplane in the incorrect direction.

$$(w^*, b^*) = \underset{w,b}{\text{argmax}} \frac{2}{\|w\|} = \underset{w,b}{\text{argmin}} \frac{\|w\|}{2}$$

$$w^*, b^* = \underset{w,b}{\text{argmin}} \frac{\|w\|}{2} + C \cdot \frac{1}{n} \sum_{i=1}^{n} \zeta_i \qquad \text{s.t. } y_i(w^T x_i + b) \geq 1 - \zeta_i \;\; \forall i, \;\; \zeta_i \geq 0$$

$$w^*, b^* = \underset{w,b}{\text{argmin}} \underbrace{\frac{\|w\|}{2}}_{\text{regularizn}} + C \cdot \underbrace{\frac{1}{n} \sum_{i=1}^{n} \zeta_i}_{\text{hinge loss}}$$

1/margin

avg distance from incorrect hyperplane for misclassified points

C is the hyperparameter here.

C behaves like $\frac{1}{\lambda}$ → hyperparameter in logistic regression.

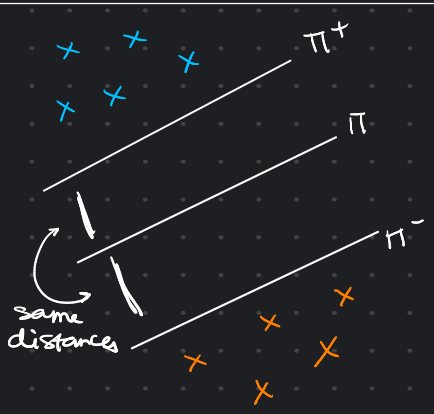
As C increases, loss term will weigh more,
tendency to make mistakes (on Dtrain) will decrease.
: overfitting (high variance model)

As C decreases, regularization term will weigh more.
: underfitting (high bias model)

This classifier is called a soft-margin classifier.

Why +1 and -1 for support vector planes?

We haven't assumed that w is a unit vector.
It could be any vector perpendicular to all the 3 hyperplanes.



margin = $d(\pi+, \pi-) = \frac{2}{\|w\|}$ $\|w\|$ need not be 1.

Task: maximize margin (with some constraints)

1st approach: $\pi^+ : w^T x + b = k \qquad k \geq 0$

$\pi^- : w^T x + b = -k$

margin = $\frac{2k}{\|w\|}$. So maximizing $\frac{2}{\|w\|}$ would give same w^*, b^* that maximizing $\frac{2}{\|w\|}$ does.

So any k is fine.

2nd approach: $\pi^+ : w^T x + b = k$

$$\left(\frac{w}{k}\right)^T x + \left(\frac{b}{k}\right) = 1$$

$$(w')^T x + b' = 1$$

w' would still be perpendicular to π. $\|w\|$ changes, but we don't care. It need not be 1.

So any values of k are fine.

1 and -1 is just convenience.

Loss minimization: hinge loss



$= y_i(w^T x + b) = z_i$

$z_i > 0$: x_i is correctly classified.

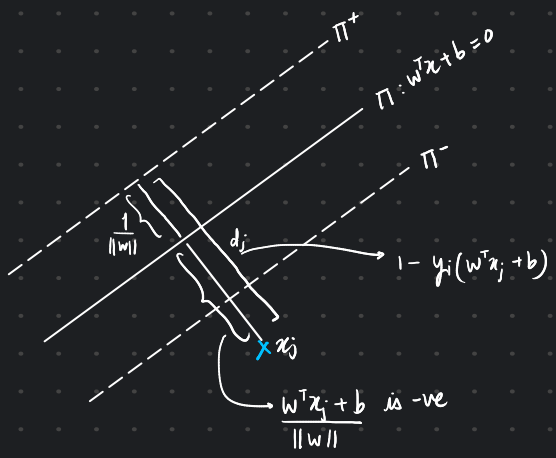
$z_i < 0$: x_i is misclassified.

$z_i \geq 1$: hinge loss $= 0$

$z_i < 1$: hinge loss $= 1 - z_i$

OR

hinge loss $= \max(0, 1 - z_i)$



z_j

$d_j = 1 - y_j(w^T x_j + b) = 1 - z_j$

ζ_j = dist for x_j to $\pi^+ = d_j = 1 - z_j$

$\zeta_j = 1 - z_j$ (when x_j is misclassified)

For a correctly classified point, $\zeta_i = 0$

So $\zeta_i = \max(0, 1 - z_i)$

Soft SVM formulation: $\min_{w,b} \frac{\|w\|}{2} + C \cdot \sum_{i=1}^{n} \zeta_i$ s.t. $1 - y_i(w^T x_i + b) \leq \zeta_i$, $\zeta_i \geq 0$

$C \uparrow$: overfitting

loss minimization: $\min_{w,b} \sum_{i=0}^{n} \max(0, 1 - y_i(w^T x_i + b)) + \lambda \|w\|^2$

$\lambda \uparrow$: underfitting

minimizing $\|w\|$ is same as minimizing $\|w\|^2$ as $\|w\| \geq 0$

Both of those represent the same thing.

$\left\{ \min_{w,b} \frac{1}{2}\|w\| + c \cdot \sum_{i=1}^{n} \zeta_i \text{ s.t. } y_i(w^T x_i + b) \geq 1 - \zeta_i \ \forall i \text{ and } \zeta_i \geq 0 \right\}$ is called the primal formulation of SVM.

Dual form

$$\max_{\alpha_i} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j x_i^T x_j \quad \text{s.t. } C \geq \alpha_i \geq 0, \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

(I) $\forall x_i$'s, there's a corresponding α_i.

(II) Here, x_i's only occur in the form $x_i^T x_j$.

(III) $f(x_q) = \sum_{i=1}^{n} \alpha_i y_i x_i^T x_q + b$

↳ can also be written as x_i's and α_i's.

(IV) $\alpha_i > 0$ only for support vectors.

$\alpha_i = 0$ for non-SVs.

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i x_i^T x_q + b$$

$\alpha_i = 0$ for non-SVs.

So only the SVs matter to compute $f(x_q)$.

$$\max_{\alpha_i} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \underline{\underline{x_i^T x_j}} \quad \text{s.t. } \alpha_i \geq 0 \quad \begin{cases} = 0 \text{ for non-SVs} \\ > 0 \text{ for SVs.} \end{cases} \quad \sum_{i=1}^{n} \alpha_i y_i = 0$$

$x_i^T x_j = x_i \cdot x_j = \text{cosine_sim}(x_i, x_j)$ if $||x_i|| = ||y_j|| = 1$ [Data is unit-normalized]

$\downarrow$

$\text{sim}(x_i, x_j)$

So the dual form is used when we have a similarity matrix.

$\text{sim}(x_i, x_j)$ is represented by $K(x_i, x_j)$.

↳ kernel func^n.

Kernel Trick

$$\max_{\alpha_i} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

We replace $x_i^T x_j$ by $K(x_i, x_j)$.
(Kernel SVM as opposed to linear SVM)

Linear SVM tries to maximize the margin in the space of xi's.
(like logistic regression tries to minimize loss in the space of xi's.

A hyperplane can sometimes be a terrible separator.
Logistic regression uses feature transforms to operate on such data.
It would find a hyperplane not in the space of xi's, but in some transformed space.

A kernel SVM with an appropriate kernel would work with such data. Kernel SVM would try to find a hyperplane in a different space as well.
The kernel trick isn't the same as function transformations, but work on a similar idea.

There's kernel logistic regression as well, that uses kernelization.

Polynomial Kernel



In logistic regression, we transformed the space $(f_1, f_2) \rightarrow (f_1^2, f_2^2)$ for this example.

$$K(x_1, x_2) = (x_1^T x_2 + c)^d$$

e.g. $K(x_1, x_2) = (1 + x_1^T x_2)^2$ $\quad c = 1,\ d = 2$ (a quadratic kernel)

say $x_1 = \langle x_{11}, x_{12} \rangle$, $x_2 = \langle x_{21}, x_{22} \rangle$

$$K(x_1, x_2) = (1 + x_{11} x_{21} + x_{12} x_{22})^2$$

$$= (1 + x_{11}^2 x_{21}^2 + x_{12}^2 x_{22}^2 + 2 x_{11} x_{21} + 2 x_{12} x_{22} + 2 x_{11} x_{21} x_{12} x_{22}$$

let $[1, x_{11}^2, x_{12}^2, \sqrt{2} x_{11}, \sqrt{2} x_{12}, \sqrt{2} x_{11} x_{12}] : x_1'$

$[1, x_{21}^2, x_{22}^2, \sqrt{2} x_{21}, \sqrt{2} x_{22}, \sqrt{2} x_{21} x_{22}] : x_2'$

$$K(x_1, x_2) = (x_1')^T (x_2')$$

$x_1 \rightarrow x_1'$, $x_2 \rightarrow x_2'$, $x_1^T x_2 \rightarrow (x_1')^T (x_2')$

Kernel trick implicitly transforms from d-dim space to some $d' > d$.

$d \rightarrow d'$

Radial Basis Function (RBF) → due to similarity with KNN.

This is a really general purpose kernel.

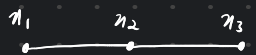
$$K_{RBF}(x_1, x_2) = \exp\left\{\frac{-\|x_1 - x_2\|^2}{2\sigma^2}\right\}$$

$$\|x_1 - x_2\|^2 \to d_{12}^2 \qquad \overset{d_{12}}{\underset{x_1 \quad x_2}{---}}$$

$\sigma \to$ hyperparameter

$$= \exp\left\{\frac{-d_{12}^2}{2\sigma^2}\right\} = \frac{1}{e^{d^2/2\sigma^2}}$$

① As $d_{12} \uparrow$, $K(x_1, x_2) \downarrow$ ∴ behaves like similarity.

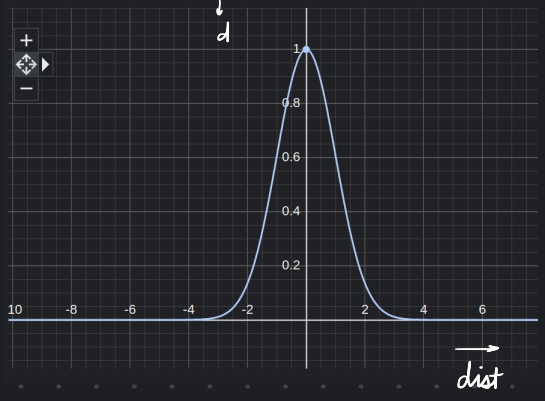


$K(x_1, x_2) > K(x_1, x_3)$

② Effect of σ:

plot $\sigma = 1$, $\sigma = 0.1$, $\sigma = 10$

Graph for $\exp(-x^2/(2 \cdot 1))$ $\quad e^{-x^2/2\cdot 1}$ $\quad \sigma = 1$



ⓘ $d = 0 \to K = 1$

ⓘⓘ $|d| \uparrow$ $K \downarrow\downarrow$ exponentially

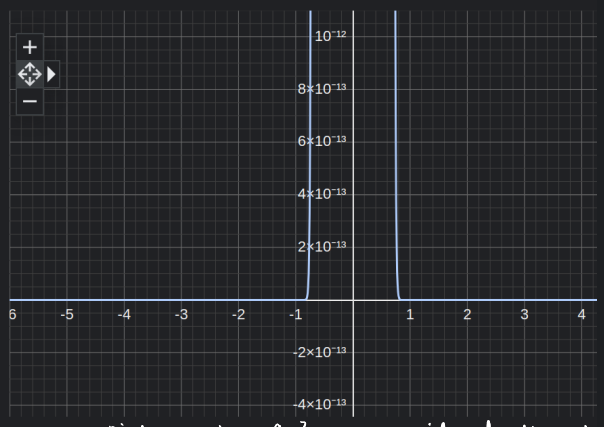
As points get further, $K \to 0$

As dist $\to \infty$, similarity $\to 0$

$K \to 0$.

Distances > 4 are considered dissimilar.

Graph for $\exp(-x^2/(2 \cdot 0.01))$ $\quad \sigma = 0.1$



Distances > 0.9 ? are considered dissimilar.

Graph for $\exp(-x^2/(2 \cdot 100))$ $\quad \sigma = 10$



Distances > 9.∞ ? ish are considered dissimilar.

So as σ becomes larger, we consider a higher # of points similar.
(More points that are far away are considered similar).

Like in K-NN, the nearest K points are considered. So a large K means a large # of points are considered similar.

$\sigma \uparrow \Rightarrow K \uparrow$

(RBF) (KNN)

RBF-SVM is a good approximation for KNN.
Use grid CV / random CV to figure out the hyperparameters.

Use RBF kernel if you don't know the perfect one.

Domain Specific Kernels

Feature transformation essentially works on a lot of domain-specific knowledge. As SVMs became more popular, a lot of domain-specific kernels were figured out.
Examples:
String kernels for measuring similarity between pairs of strings,
Genome kernels for genome prediction,
Graph kernels for computing inner products on graphs, measuring similarity between pairs of graphs.

Train and runtime complexity

Training:

It can be done using stochastic gradient descent (SGD). There are specialized algos to solve the dual SVM problem more efficiently like the sequential minimal optimizer (SMO).
(libSVM is an open-source library for training SVMs)

Since we do not know how many iterations an optimization problem would require, we cannot tell the exact time complexity.
Approximately,

Train time = $O(n^2)$ for kernel SVMs. There's an algo from 2007 that takes $O(nd^2)$ time if $d<n$.

↳ If n is large, SVM isn't used. } this is changing though.

Runtime:

$$f(x_q) = \sum_{i=1}^{n} \alpha_i y_i K(x_i, x_q) + b$$

$\alpha_i = 0$ for non-support vectors.
So only the values for support vectors matter.

So finding y_q from an x_q depends on # of support vectors.

$T = O(kd)$ k = # of support vectors } $\in [1, n]$
d = dimensionality

Here, the # of support vectors could be anything upto n. We have no control over it.
So O(kd) could be really large or really small.

In logistic regression, runtime complexity is O(d) which is usually smaller.

In nu-SVM, we can know this a little.

nu-SVM

In the standard SVM formulation (c-SVM), we only place the condition c >= 0 on the hyperparameter c.

In nu-SVM, 0 <= nu <= 1.

nu >= fraction of errors
nu <= fraction of SVs.

E.g. Say we want to build an SVM from D_{train} with no more than 10% error.
Then $\nu = 0.1$
for < 1% errors, $\nu = 0.01$

If we say $\nu = 0.01$, then % of errors $\leq$ 1%
of support vectors $\geq$ 1% of n] -> again, we cannot control the # of support vectors.
They can be anything $\geq$ 1% of n

nu-SVM helps us control the error
and helps us get an idea about the # of SVs.

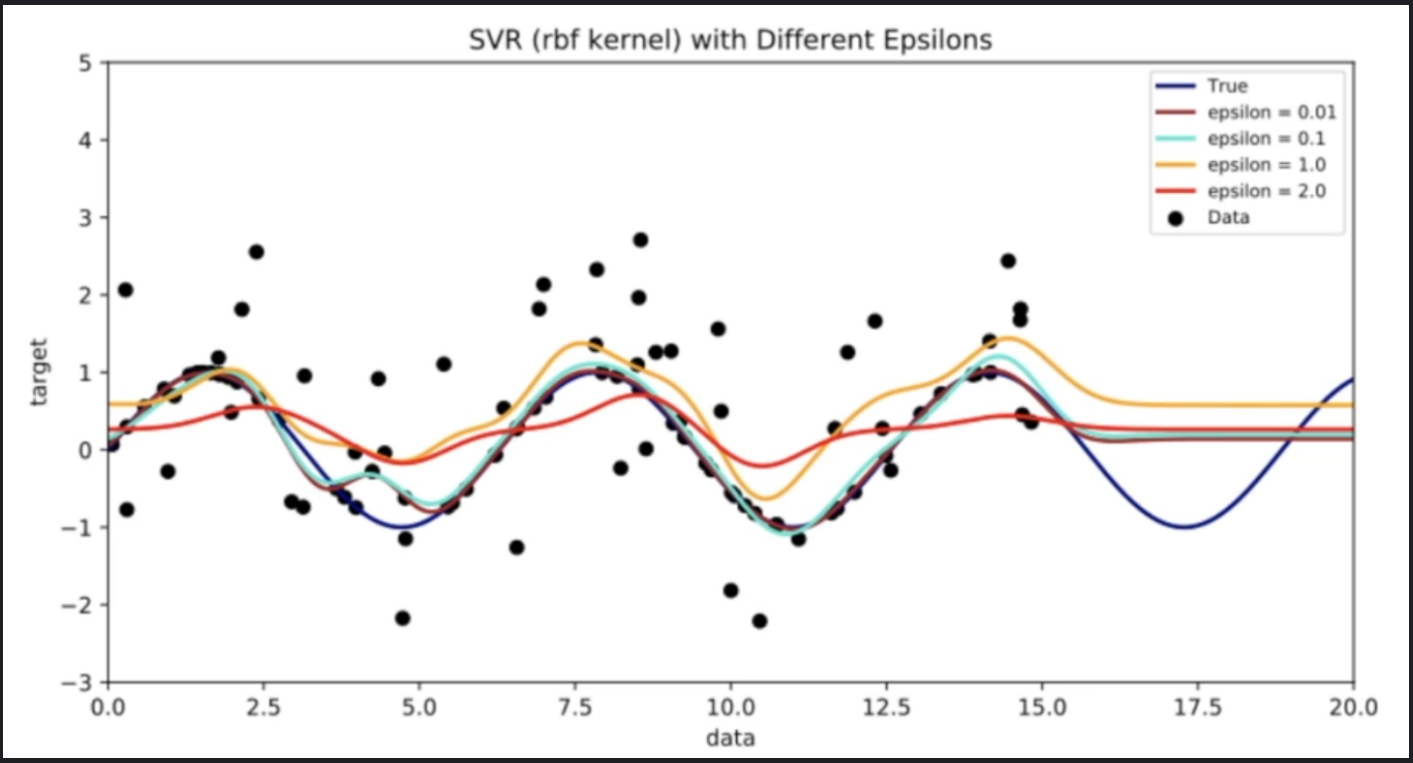
Support Vector Regression

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad s.t.\ y_i - (w^T x_i + b) \le \epsilon \quad (w^T x_i + b) - y \le \epsilon \quad \forall i,\ \epsilon \ge 0$$

i.e. $y_i - \hat{y}_i \le \epsilon$, $\hat{y}_i - y_i \le \epsilon$.

$\epsilon \downarrow \Rightarrow$ errors on training data $\downarrow\downarrow \Rightarrow$ overfitting $\uparrow\uparrow$

$\epsilon \uparrow \Rightarrow$ errors on D_{train} $\uparrow\uparrow \Rightarrow$ underfitting $\uparrow\uparrow$



Cases

Feature transformation ⟶ Kernel design

Decision surface:
- linear SVM: hyperplane
- kernel SVM: non-linear in original space, but linear in higher dim space

Similarity function can be converted to kernel function with a few modifications.

Feature importance and interpretability is difficult.
Forward feature selection can be used.

Outliers: They have negligible impact on an SVM, since only support vectors matter.
In rbf with a small σ (σ is like k in KNN), you could overfit to the outliers.

Bias-variance tradeoff:

c↑ ⇒ overfitting ⇒ high variance

c↓ ⇒ underfitting ⇒ high bias

d (dim) is high:
Good.
Kernel SVM tries to take data to a higher dim. So having d high is good.

Best case:
- right kernel (works like magic)

Worst case:
- n is large ⟶ train time is high (can't use for low latency purposes)
- k is large ⟶ low latency is NOT possible (time to calculate f(Xq) is high) (# of SVs)